

Credit Risk Intelligence Platform

An end-to-end system that scores loan applicants, explains every score in plain English, distils the model into rules a policy team can read, and lets analysts ask the data questions directly — built on Home Credit's 307,511-application dataset.

DATASET

Home Credit Default Risk · 7 tables

MODEL

LightGBM · ROC-AUC 0.7695

STACK

FastAPI · React · PostgreSQL · Docker

AUTHOR

Jash Kevadiya

Faster credit decisions still need to be explainable ones

Banks can't trade accuracy for a black box — every one of these needs has to be satisfied at once, not traded off against the others.

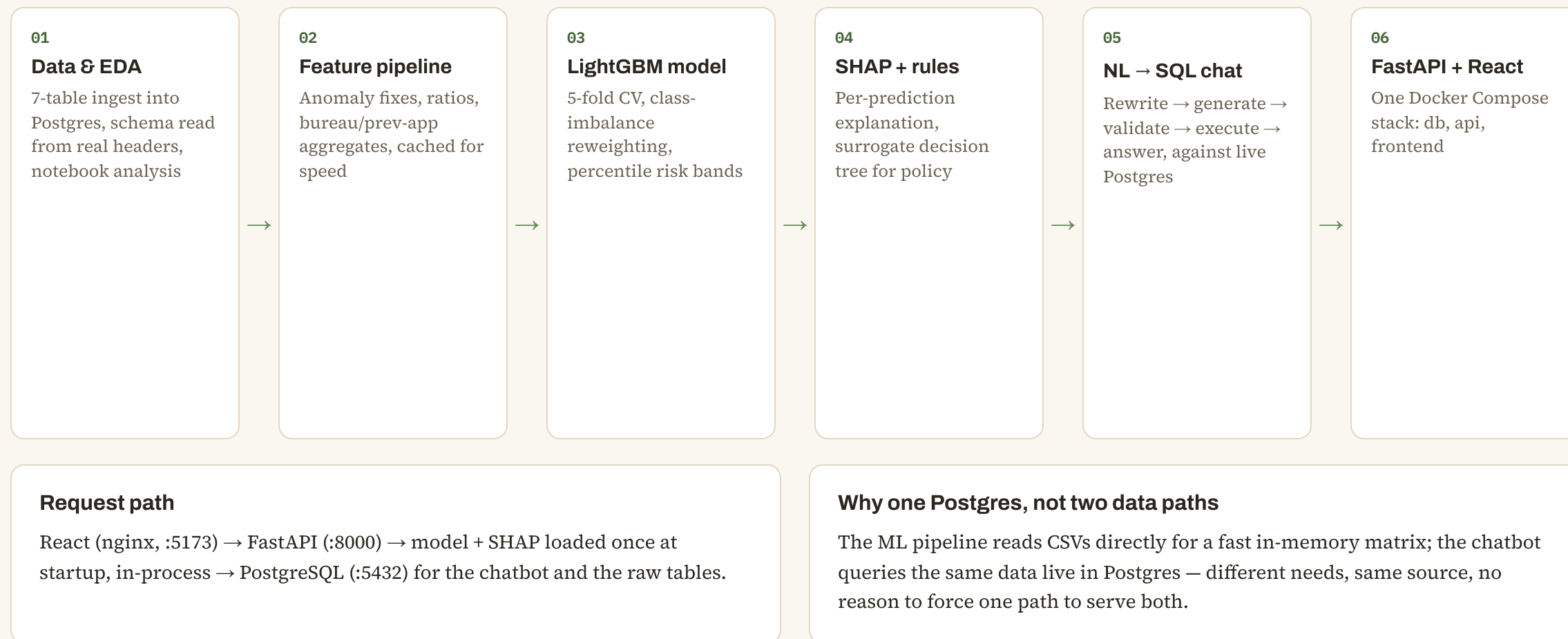
- 1 **Identify high-risk applicants early** — before capital is committed, not after a default.
- 2 **Automate scoring and decision support** — a consistent score for every applicant, not case-by-case judgment.
- 3 **Explain every classification** — a rejected applicant, an auditor, or a regulator can all ask "why," and the answer can't be "the model said so."
- 4 **Satisfy audit and regulatory expectations** — every prediction and every generated query is logged and re-checkable, not a one-off black-box call.
- 5 **Let analysts explore data in plain English** — without waiting on someone who knows SQL.
- 6 **Bridge ML insight and credit policy** — a model that only a data scientist can read never becomes a policy.

8.07%

of applicants in this dataset had payment difficulty — rare enough that a naive model can look 92% "accurate" while catching almost none of them.

One pipeline, six real stages

Every stage below is a real, running module — not a slide-only diagram.



What the data actually says

Six findings that came straight out of the notebook — and that later shaped the model, the rules, and the chatbot's schema.

307,511

Applicants analyzed

122

Raw features

8.07%

Had payment trouble

67

Columns with missing data

Six findings that shaped the build

1. External credit scores (EXT_SOURCE 1–3) are the strongest single predictor.
2. Younger applicants have more payment trouble than older ones.
3. Income type and education level both separate risk cleanly.
4. Loan-to-income ratio matters, but only as one signal among several.
5. **DAYS_EMPLOYED** placeholder flags a *lower*-risk group (retirees).
6. Zero credit-bureau history is itself a mild risk signal, not a neutral "unknown."

Credit Risk Intelligence Platform

Home Credit Default Risk: exploratory analysis, risk scoring, explainability, policy rules, and a talk to data assistant.

EDA
Explore the data

Risk Prediction
Score and explain an applicant

Decision Rules
The model's policy

Chatbot
Ask in plain English

Built end to end: data pipeline, ML model, explainability, business rules, and a talk to data chatbot, all working on the real dataset.

Exploratory Data Analysis

A first look at who is applying for these loans, what shape the data is in, and the patterns that matter most for spotting risk. Computed once in the analysis notebook and shown here as is.

How this was built (in plain terms): We loaded every loan application into a table, checked which pieces of information were missing or unusual, and drew charts to spot patterns before building any prediction model. Nothing here uses AI, it is careful counting and charting of the real data.

307,511

Applicants

122

Features

8.07%

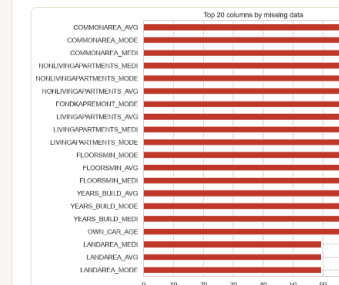
Had payment trouble

67

Columns with missing data

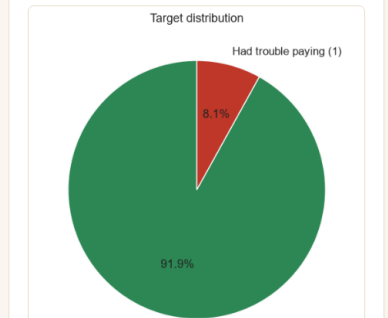
Most missing data is in optional building/apartment fields

The top missing-value columns are almost entirely apartment/building details. Expected, since many applicants rent or live with family and simply have nothing to report there, not a data error.



Only ~8% of applicants had trouble paying

A real class imbalance. A lazy model could predict 'no trouble' every time and still look 92% accurate while being useless. Drives the class-imbalance handling in the model.



LightGBM, chosen and proven against a baseline

Every number below is from 5-fold cross-validated, out-of-fold predictions — never scored on data the model trained on.

MODEL	ROC - AUC
Logistic Regression (baseline)	0.7572
LightGBM (deployed)	0.7695

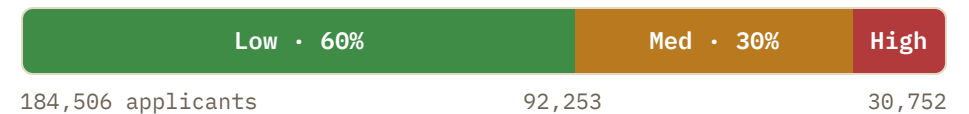
METRIC	VALUE
PR-AUC	0.2557
KS statistic	0.402
F1 @ threshold 0.67	0.3169
Recall @ threshold	40.6%

Class imbalance: reweight, don't resample

`scale_pos_weight = 11.39` — the real ratio of non-defaulters to defaulters, not a tuned guess. Reweights the loss function instead of synthesising (SMOTE) or discarding (undersampling) real rows.

Risk bands — percentile, not fixed cutoffs

Fixed cutoffs (e.g. 0.20) put ~75% of applicants in "High" once probabilities were reweighted — percentiles stay meaningful regardless of calibration.

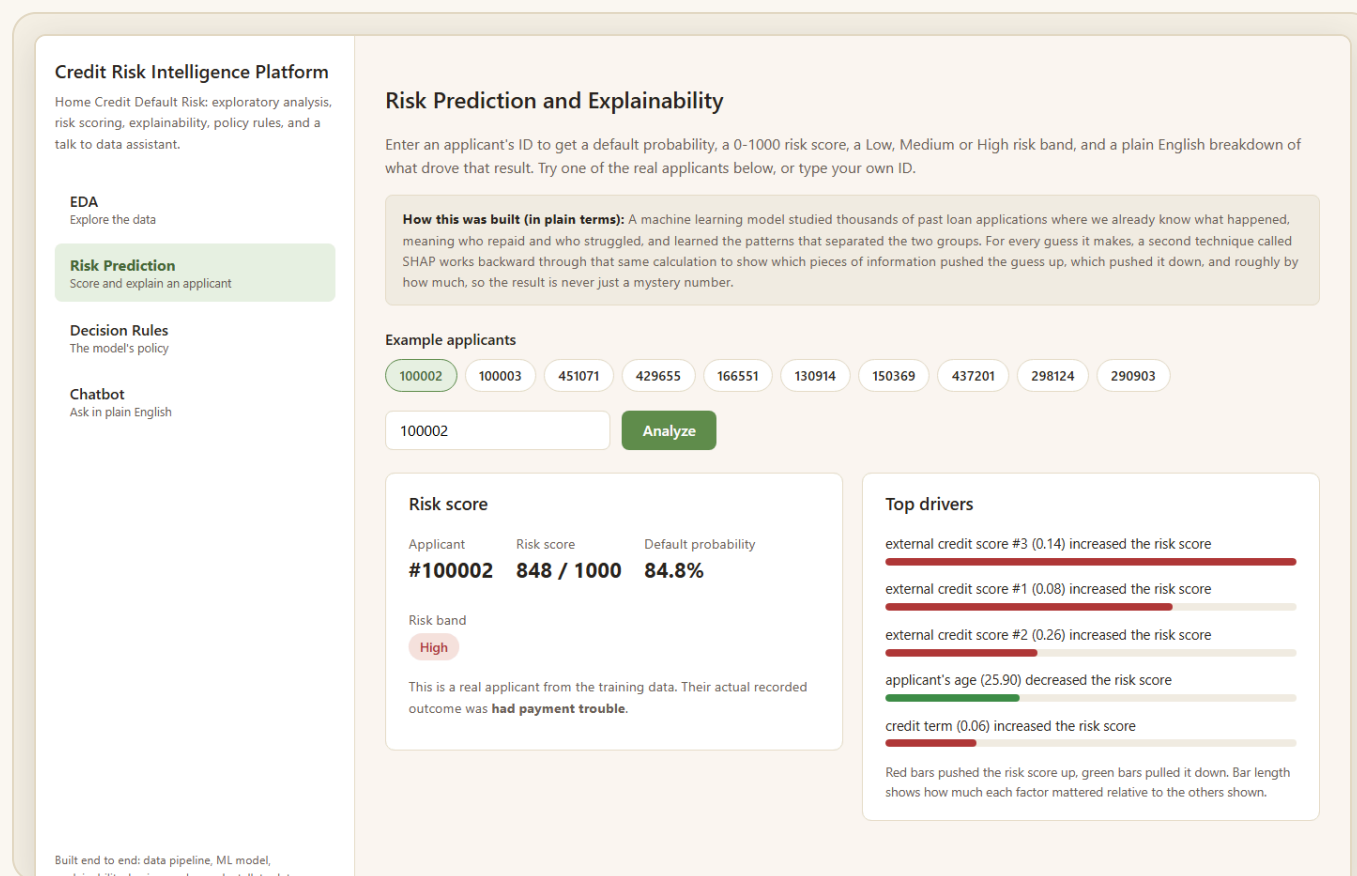


Why LightGBM over XGBoost

A convenience call, not a capability gap — both now handle categoricals and histogram splitting natively. LightGBM does both **by default** for this dataset's mix of categorical and high-cardinality columns.

Every score comes with its reasons

SHAP's TreeExplainer breaks down the model's own calculation for one specific applicant — exact for tree ensembles, no approximation.



Applicant #100002

Scored 848 / 1000 — **High** risk. Their real recorded outcome: had payment trouble.

All three external credit scores pushed the risk up; their age (25.9) pulled it slightly down — a genuine feature interaction, not a contradiction of the population trend.

Why only the top 5 factors

SHAP computes a value for all ~134 features internally — showing all of them to a credit officer would defeat the point of "explainable." Feature names are mapped to plain English, shared with the rules module.

A model a credit committee can actually read

A depth-4 surrogate tree is fit to the LightGBM model's own predictions, not the raw labels — it describes what the deployed model does.

RULE (ABRIDGED)	DEFAULT RATE	LIFT	SUPPORT
Both external scores very low (≤ 0.21 / ≤ 0.27)	33.6%	4.16×	4,378
Ext. score 3 mid, score 2 low (≤ 0.16)	22.8%	2.82×	9,076
Ext. score 3 mid, score 2 mid (0.16–0.47)	12.7%	1.57×	36,376
Both external scores high, goods ratio moderate	4.5%	0.56×	29,882
Both external scores very high (> 0.65)	2.1%	0.25×	30,501

Honest limitation, stated up front

13 of 16 rules split only on the two external credit scores — they dominate the model's decision surface. Confirmed independently by EDA and SHAP, not a rule-miner artifact.

Credit Risk Intelligence Platform

Home Credit Default Risk: exploratory analysis, risk scoring, explainability, policy rules, and a talk to data assistant.

EDA

Explore the data

Risk Prediction

Score and explain an applicant

Decision Rules

The model's policy

Chatbot

Ask in plain English

Built end to end: data pipeline, ML model, explainability, business rules, and a talk to data chatbot, all working on the real dataset.

Decision Rules

The model's overall logic, distilled into a small set of plain "if this, then that" statements. A static summary a credit policy team could review on its own, without needing to run any prediction. Sorted from highest to lowest observed risk.

How this was built (in plain terms): We asked a much simpler, easy to read version of the model to copy the behaviour of the real, more complicated model. That simpler version naturally organises itself into a short list of plain rules, which is what you see below. Each rule also shows how many real applicants fall into it and what actually happened to them, so the rules can be checked against reality, not just trusted blindly.

IF external credit score #3 <= 0.48 AND external credit score #2 <= 0.47 AND external credit score #3 <= 0.27 AND external credit score #2 <= 0.21 THEN observed default rate is 33.6% (4,378 applicants, 4.16x the average rate)

IF external credit score #3 <= 0.48 AND external credit score #2 <= 0.47 AND external credit score #3 > 0.27 AND external credit score #2 <= 0.16 THEN observed default rate is 22.8% (9,076 applicants, 2.82x the average rate)

IF external credit score #3 <= 0.48 AND external credit score #2 <= 0.47 AND external credit score #3 <= 0.27 AND external credit score #2 > 0.21 THEN observed default rate is 21.7% (8,869 applicants, 2.69x the average rate)

IF external credit score #3 <= 0.48 AND external credit score #2 > 0.47 AND external credit score #3 <= 0.27 AND external credit score #1 <= 0.58 THEN observed default rate is 15.1% (16,382 applicants, 1.87x the average rate)

IF external credit score #3 > 0.48 AND external credit score #2 <= 0.50 AND years at current job <= 8.07 AND external credit score #2 <= 0.23 THEN observed default rate is 13.0% (7,177 applicants, 1.61x the average rate)

IF external credit score #3 <= 0.48 AND external credit score #2 <= 0.47 AND external credit score #3 > 0.27 AND external credit score #2 > 0.16 THEN observed default rate is 12.7% (36,376 applicants, 1.57x the average rate)

Plain English in, grounded answers out

Question → rewrite → SQL → validate → execute → answer — five real query patterns, three independent safety layers.

Credit Risk Intelligence Platform
Home Credit Default Risk: exploratory analysis, risk scoring, explainability, policy rules, and a talk to data assistant.

EDA
Explore the data

Risk Prediction
Score and explain an applicant

Decision Rules
The model's policy

Chatbot
Ask in plain English

Or rankings and means (e.g. the 10 highest earners who had payment difficulty)
4. Ratios between two groups (e.g. loan to income ratio, defaulted vs not)
5. Questions spanning multiple tables (e.g. applicants with bad debt on their credit bureau record)

How this stays fast and cheap to run (token optimization):

- The AI only ever sees a compact list of table and column names, not long descriptions of every field, so each request stays small.
- Query results are capped at 200 rows by the database, and only the first 50 are ever shown to the step that writes the final answer, so one huge result never balloons a request.
- Follow-up questions are first condensed into a single, self-contained question by a small dedicated step, instead of replaying the entire conversation into every request as the chat grows longer.

What is the default rate by education level?

Lower secondary: 10.93%
Secondary / secondary special: 8.94%
Incomplete higher: 8.48%
Higher education: 5.36%
Academic degree: 1.83%

▼ Show SQL and raw rows (5 rows)

```
SELECT name_education_type, AVG(target) AS default_rate, COUNT(*) AS n_applicants FROM
```

Try asking

What is the average annual income of all applicants? What is the default rate by education level?

Show the 10 applicants with the highest income who had payment difficulty.

What is the average loan-to-income ratio for applicants who defaulted versus those who didn't?

How many applicants have at least one bad debt record with the credit bureau?

Ask a question about the data... **Send**

Built end to end: data pipeline, ML model, explainability, business rules, and a talk to data chatbot, all working on the real dataset.

Token optimization, measured not assumed

- Full 7-table schema from Postgres, but as a compact name+type list, not prose — ~2,300 tokens.
- Results capped at 200 rows by the database; only 50 ever reach the answer step.
- A dedicated **rewrite node** condenses follow-ups into one standalone question, so history is read once — not replayed into every request.

Three independent safety layers

Prompt-level refusal + AST validation (**sqlglot**, SELECT-only, table allowlist) + a Postgres **READ ONLY** transaction. Each layer tested independently — a raw **DELETE** sent straight past the validator was still blocked by the transaction alone.

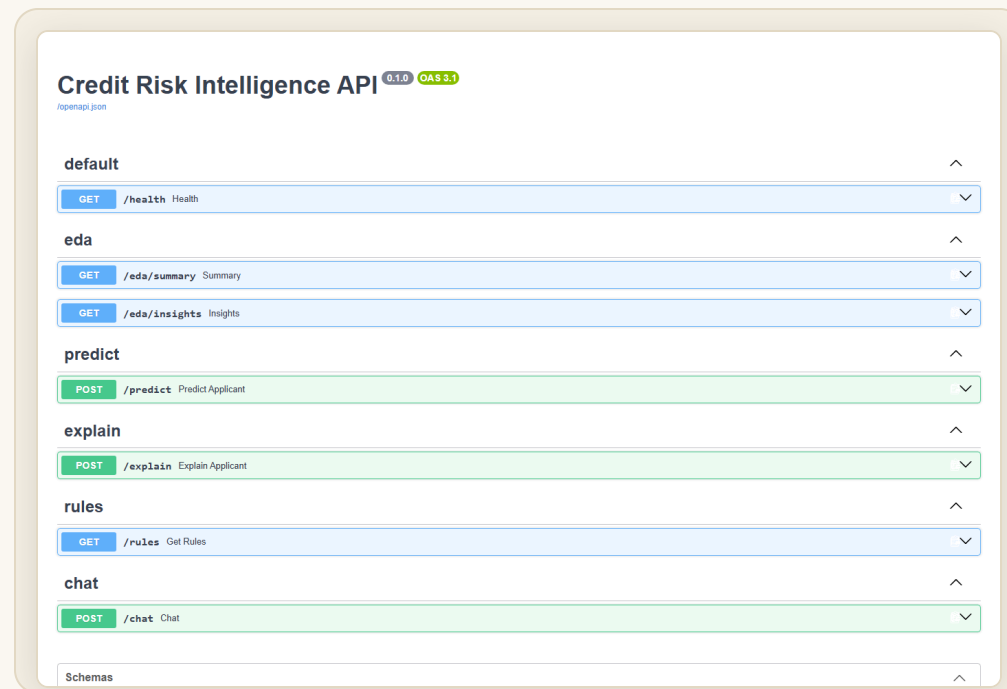
One command, three services, zero manual steps

Verified from a completely clean build — no cached state, no pre-existing volumes.

```
$ docker compose up --build
✓ credit_risk_db      healthy
✓ credit_risk_api     ingesting 7 tables → running
✓ credit_risk_frontend nginx serving on :5173
```

What happens automatically on first boot

1. Postgres creates the schema from `sql/schema.sql`
2. The API container ingests all 7 CSVs (idempotent — safe to restart)
3. The trained model and every artifact load once, in-process
4. The React frontend is served as a static build via nginx



Model + artifacts, baked in

The trained model and generated charts/rules are copied into the API image at build time — the evaluator sees real results without retraining anything.

Delivered against the brief, limitations stated plainly

Delivered

- ✓ EDA with 6+ business insights, model with cross-validated metrics and class-imbalance handling
- ✓ SHAP explainability and surrogate-tree business rules, both grounded in real observed data
- ✓ NL-to-SQL chatbot: 5 verified query patterns, 3 safety layers, conversation memory via a rewrite node
- ✓ Full-stack UI and a single-command Docker deployment, verified clean end to end

Known limitations

- No held-out test-set evaluation — `application_test.csv` has no labels; metrics are out-of-fold CV.
- No hyperparameter search beyond early-stopping-selected tree count.
- Rules lean heavily on two external scores — an honest reflection of the data, not hidden.
- Single-provider (OpenAI) chatbot rather than provider-agnostic.

Every decision above — and why it was made — is recorded in `DECISIONS.md` in the repository.